



**Tecnológico  
de Monterrey**

# **05. Repaso de ggplot Y manejo de SQL en R**

**Módulo 5. Inteligencia Artificial**

**Jorge  
Juvenal  
Campos Ferreira**

✉ [juvenal.campos@tec.mx](mailto:juvenal.campos@tec.mx)

# Programa de la clase

- Repaso de gráficas y visualizaciones
- Repaso de SQL y Bases de datos
- Ejercicios de Visualización y de SQL

# Partes del tema de una gráfica ggplot

**ggplot2**  
theme elements  
reference

Set minimal as the baseline theme:

```
theme_minimal() +  
theme(theme.element = element_type())
```

Use `element_blank()` to remove an element

Axis titles, text, ticks, and lines can be specified per axis using theme inheritance by putting `.x/.y` at the end of the theme element.

```
axis.line.y = element_line()
```

```
axis.title.y = element_text()
```

```
panel.grid.major = element_line()
```

```
panel.grid.minor = element_line()
```

```
axis.text.y
```

```
axis.text = element_text()
```

```
axis.text.x
```

```
plot.title.position = "plot"  
plot.caption.position = "plot"  
plot.title = element_text()  
plot.subtitle = element_text()
```

"plot" means that they will be aligned to the entire plot (instead of the panel)

```
plot.margin = margin(25, 25, 25, 25)
```

```
legend.title = element_text()
```

```
legend.background = element_rect()
```

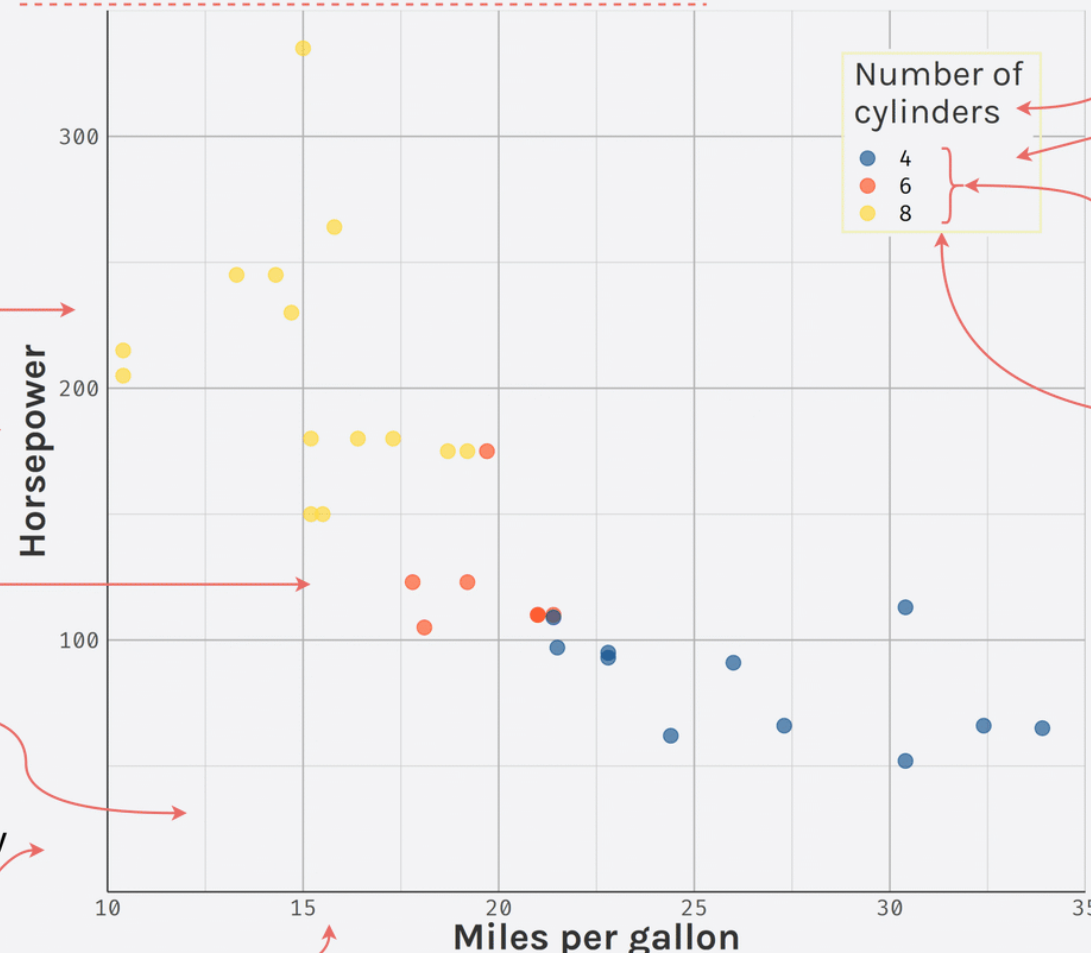
```
legend.text = element_text()
```

```
legend.position = c(.85, .85) / "none" /  
"left" / "right" /  
"bottom" / "top"
```

```
plot.background = element_rect()
```

```
plot.caption = element_text()
```

**Miles per Gallon & Horsepower**  
of 32 Automobiles(1973-74 models)



Data: 1974 Motor Trend US magazine

isabella-b

`text = element_text()` ← modifications will be applied to all text elements

Full list of elements at [ggplot2.tidyverse.org/reference/theme](https://ggplot2.tidyverse.org/reference/theme)

**Fuente:** <https://isabella-b.com/blog/ggplot2-theme-elements-reference/>

# Partes del tema de una gráfica ggplot

## ggplot2 Theme Elements

`theme(element_name = element_function())`

- `element_text()`
- `element_line()`
- `element_rect()`
- `element_blank()`

## Axis elements:

`axis.ticks`  
`element_line()`

`axis.title`  
`element_text()`

`axis.text`  
`element_text()`

`axis.line`  
`element_line()`

## Plot elements:

`plot.background`  
`element_rect()`

`plot.title`  
`element_text()`

`plot.margin`  
`margin()`

## Facetting elements:

`strip.background`  
`element_rect()`

`panel.spacing`  
`unit()`

`strip.text`  
`element_text()`

## Legend elements:

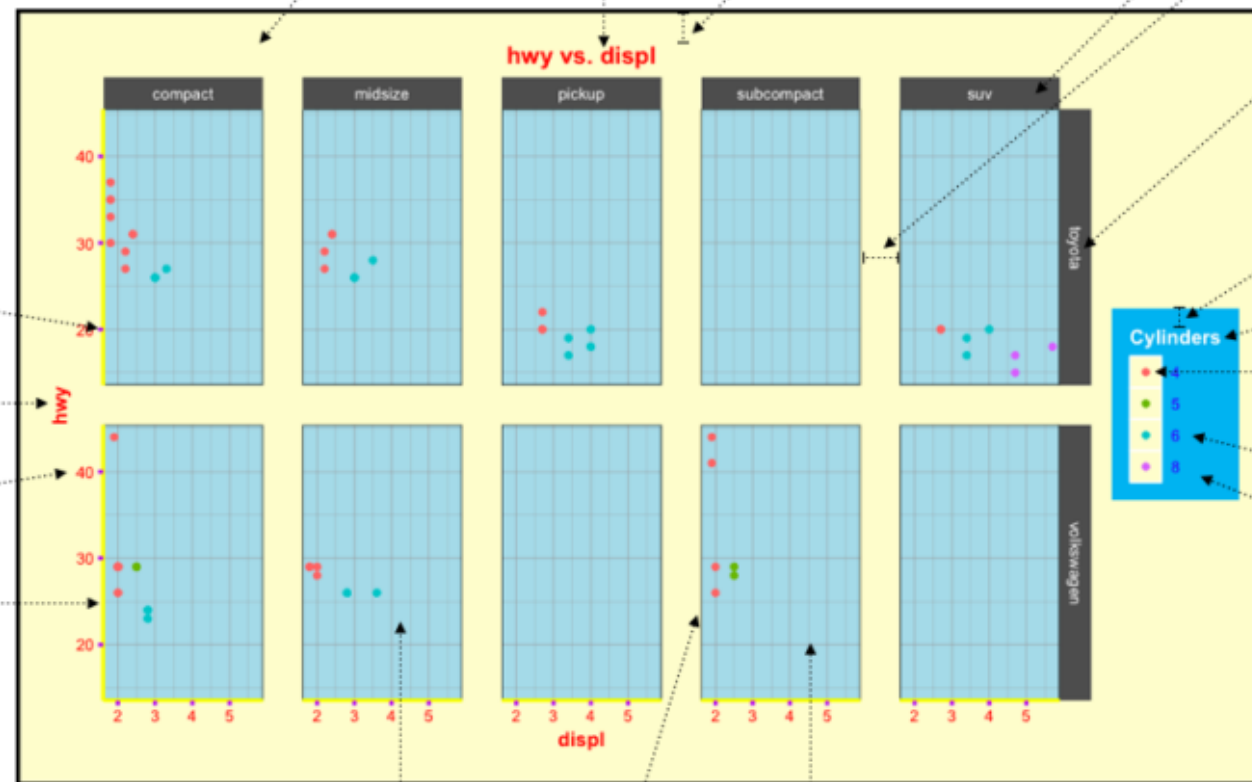
`legend.margin`  
`margin()`

`legend.title`  
`element_text()`

`legend.key`  
`element_rect()`

`legend.text`  
`element_text()`

`legend.background`  
`element_rect()`



`panel.background`  
`element_rect()`

`panel.grid`  
`element_line()`

`panel.border`  
`element_rect(fill = NA)`

## Panel elements:

[henrywang.nl](https://henrywang.nl)

Derived from "ggplot2: Elegant Graphics for Data Analysis"

## Ejercicio 01

Descargue el material de la sesión 05. En la carpeta de “Ejercicio 01” viene el script `ejercicio_01.R` con código pre-programado para realizar gráficas.

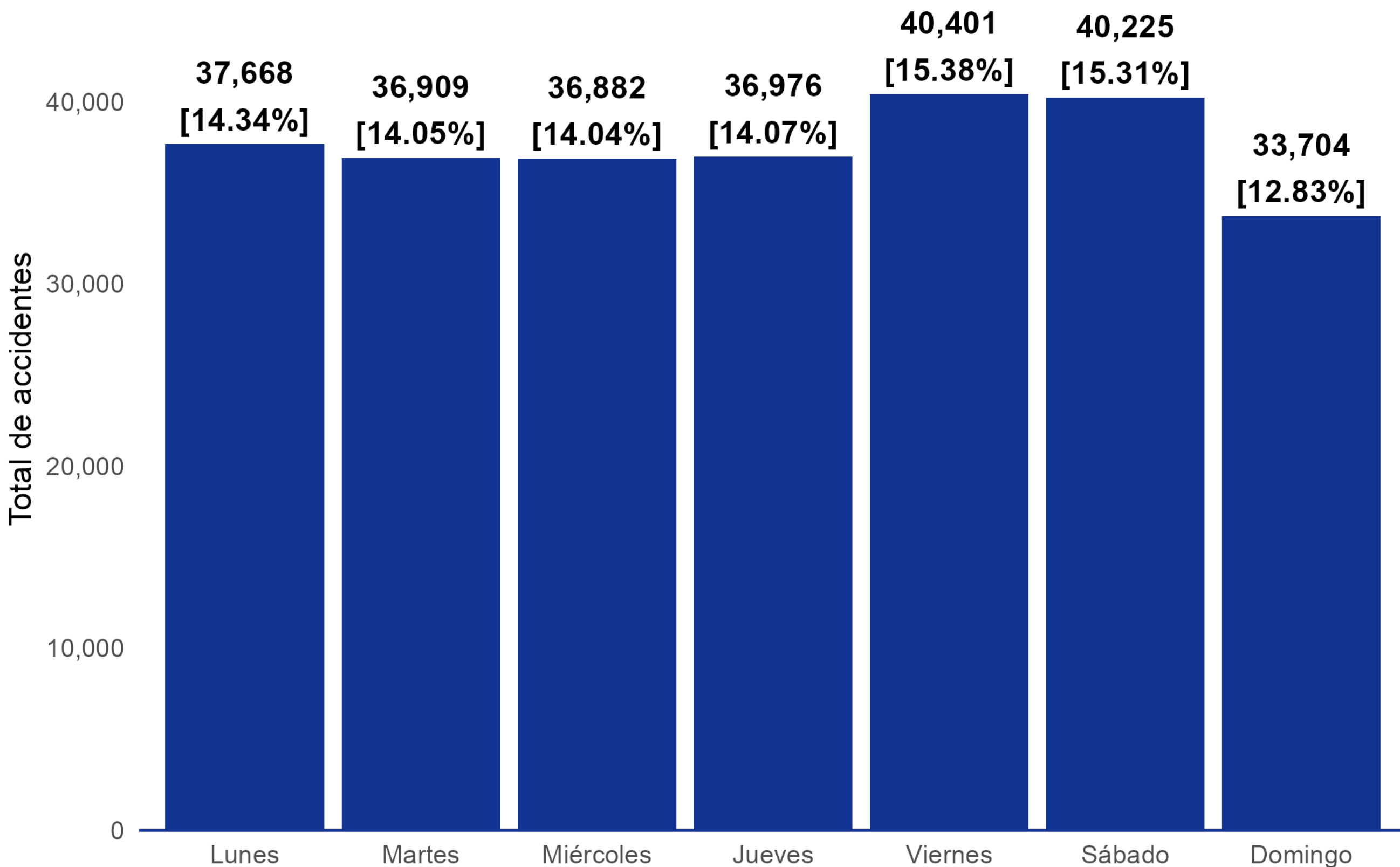
Analice que hace el código que se le proporcionó.

A partir de ese código, por favor, replique las gráficas que se ven a continuación en las diapositivas siguientes.

PLANTILLA

# Total de accidentes por día de la semana

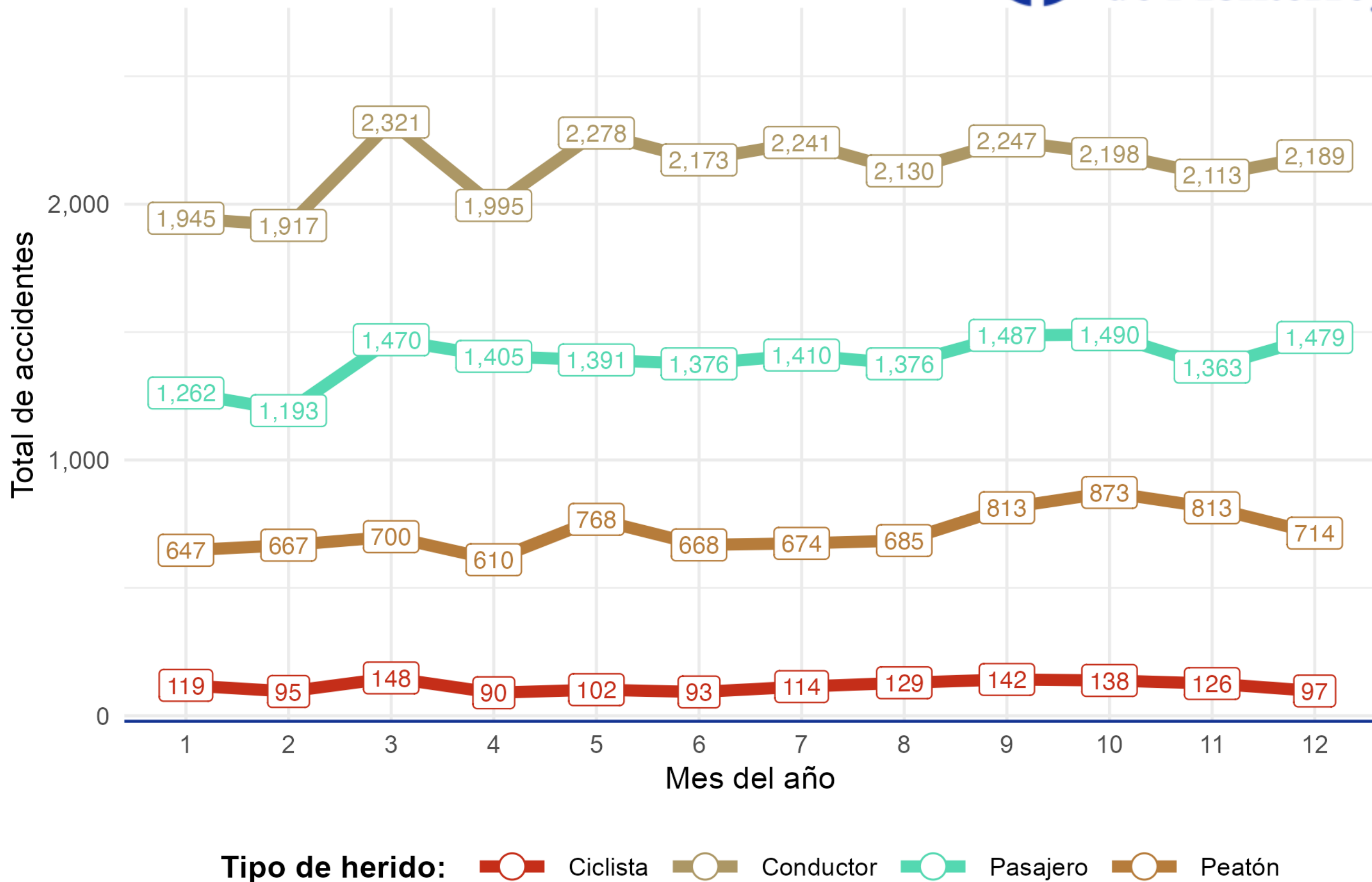
Accidentes correspondientes al año 2023





# Total de heridos, por tipo y mes del año

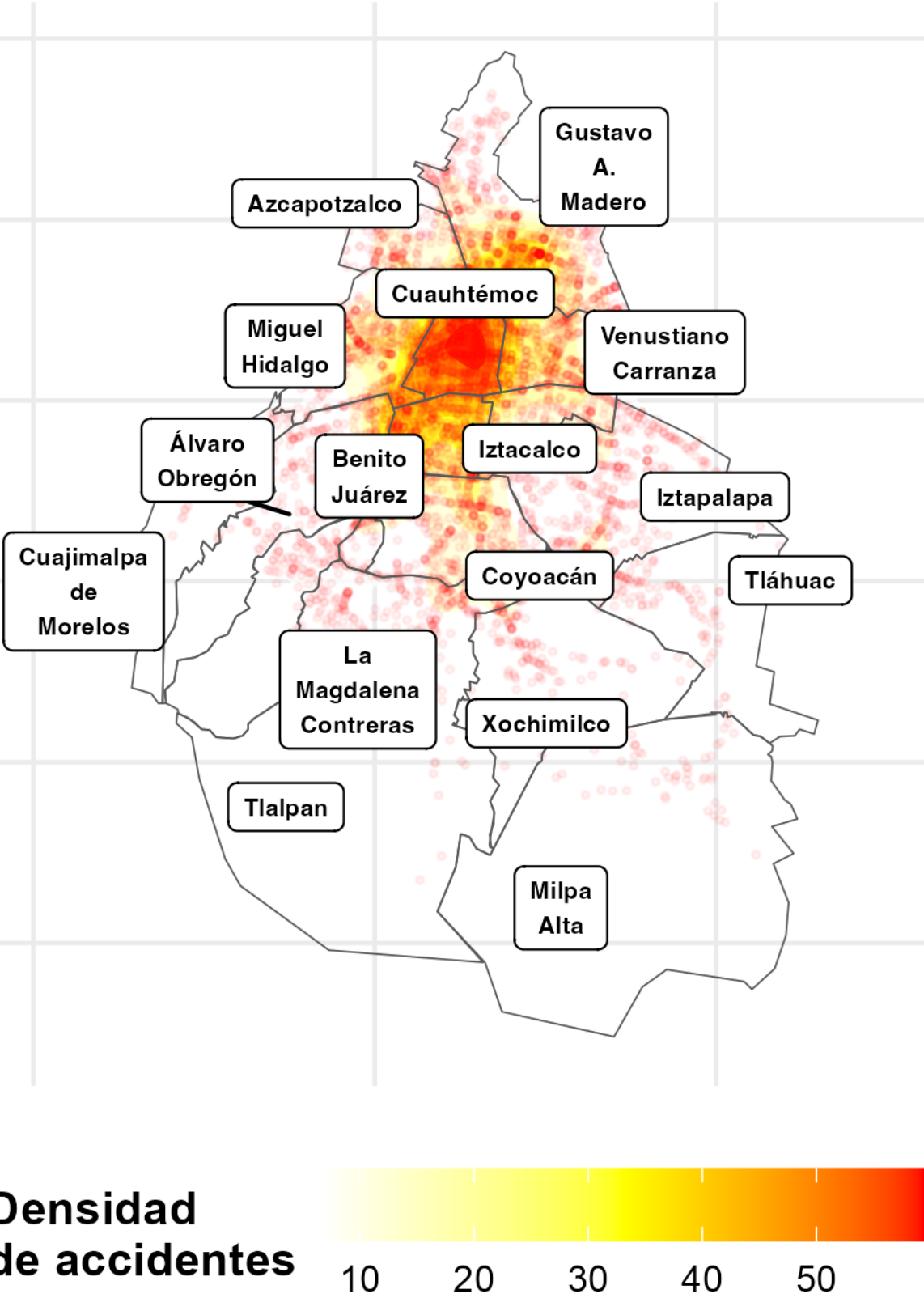
Accidentes correspondientes al año 2023





# Ubicación de accidentes registrados en el ATUS en las alcaldías de la Ciudad de México

Accidentes correspondientes al año 2023



**SQL**

# ¿Qué es SQL?



**SQL** (por sus siglas en inglés *Structured Query Language*, «lenguaje de consulta estructurada») es un lenguaje **especializado para interactuar con bases de datos relacionales**. Fue diseñado para **consultar** datos de forma eficiente y **administrar** (*insertar, modificar, borrar*) la información en sistemas de bases de datos.

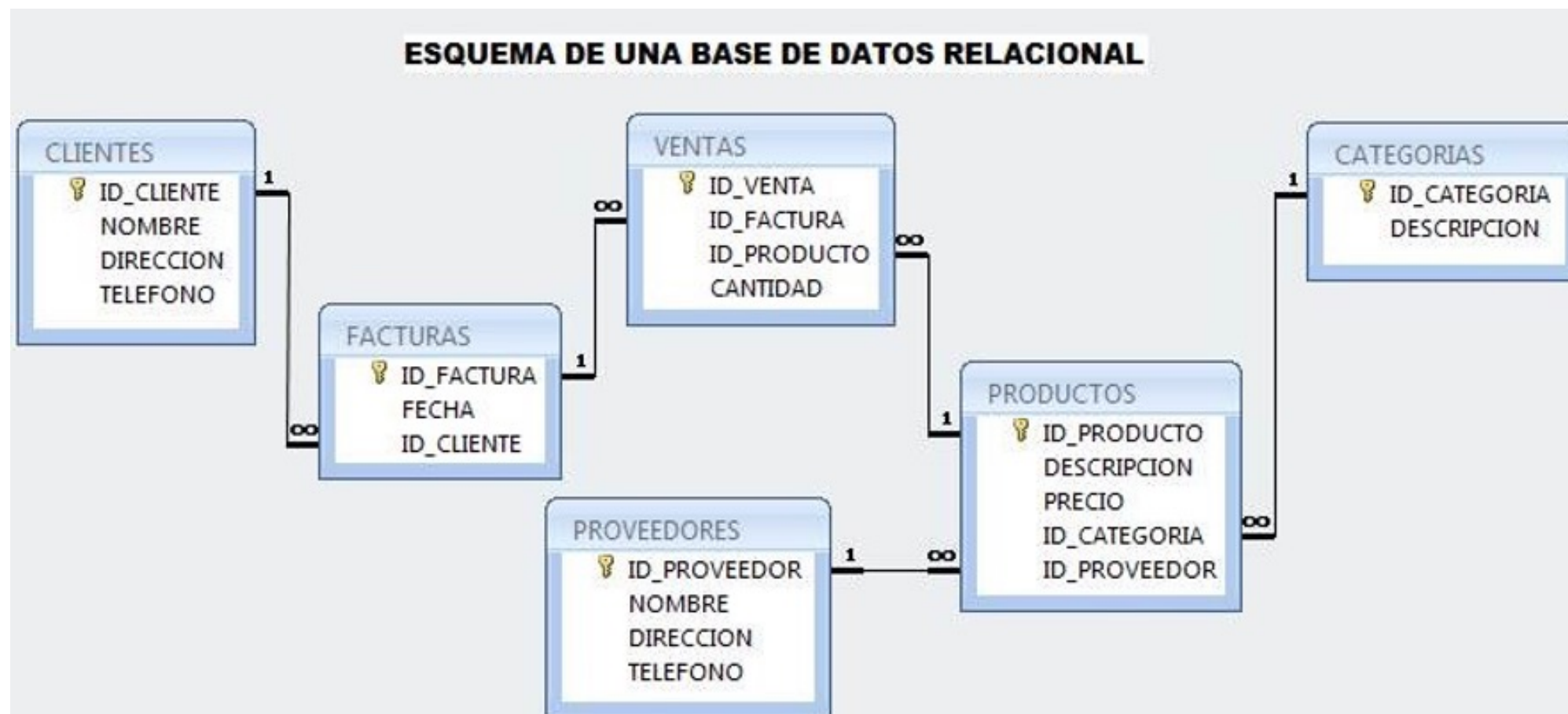
En esencia, SQL permite recuperar información de una o múltiples tablas mediante consultas declarativas (*uno especifica qué datos quiere y el sistema se encarga de cómo obtenerlos*) y realizar cambios en la base de datos, todo ello siguiendo un **estándar ampliamente adoptado desde 1986**.

**Es el lenguaje principal que utilizan prácticamente *todas* las bases de datos relacionales modernas para gestionar sus datos.**

# ¿Qué son las bases de datos relacionales?



Las **bases de datos relacionales** organizan la información en tablas compuestas por filas y columnas, permitiendo **crear relaciones entre ellas** mediante claves comunes para facilitar la consulta y el acceso a datos relacionados.



# ¿Por qué es útil SQL?



- \* **Estándar universal y fácil de aprender.** Es el estandar para gestioanr datos en aplicaciones de todo tipo.
- \* **Integracion con otros lenguajes y herramientas.** Es compatible con prácticamente cualquier lenguaje de programacion o plataforma de análisis de datos, como R, Python o Jaca.
- \* **Manejo eficiente de grandes volúmenes de datos.** Satisface la necesidad de almacenar y manejar de manera eficiente grandes volúmenes de datos.

# ¿En qué casos se usa SQL?



- \* **Empresas y grandes organizaciones que gestionan volúmenes masivos de información.**
- \* **Aplicaciones complejas y multiusuario**
- \* **Sectores intensivos en datos (finanzas, salud, sector público).** En finanzas, por ejemplo, se emplea para almacenar históricos de mercados, registros contables o indicadores económicos; en salud, para gestionar bases de datos de pacientes; en gobiernos, para bases de datos de censos, estadísticas económicas o registros ciudadanos. Estos dominios confían en SQL por su capacidad para manejar datos interconectados con rigor y rapidez. Un banco central o un ministerio de economía, por ejemplo, almacenará en bases SQL series de tiempo de indicadores (inflación, PIB, empleo) a lo largo de décadas, facilitando consultas y análisis longitudinales que serían engorrosos de otra manera.

# ¿En qué casos se usa SQL?



- \* **Empresas y grandes organizaciones que gestionan volúmenes masivos de información.**
- \* **Aplicaciones complejas y multiusuario**
- \* **Sectores intensivos en datos (finanzas, salud, sector público).** En finanzas, por ejemplo, se emplea para almacenar históricos de mercados, registros contables o indicadores económicos; en salud, para gestionar bases de datos de pacientes; en gobiernos, para bases de datos de censos, estadísticas económicas o registros ciudadanos. Estos dominios confían en SQL por su capacidad para manejar datos interconectados con rigor y rapidez. Un banco central o un ministerio de economía, por ejemplo, almacenará en bases SQL series de tiempo de indicadores (inflación, PIB, empleo) a lo largo de décadas, facilitando consultas y análisis longitudinales que serían engorrosos de otra manera.



# ¿Qué ventajas ofrece SQL sobre excel?



- **Escalabilidad y volumen:** Una base de datos SQL puede manejar **muchos más datos** que un archivo CSV o Excel típico, ya que no necesita cargar todo el conjunto en memoria. Esto permite trabajar con datasets de gigabytes o más, superando las limitaciones de RAM o de filas de una hoja de cálculo.
- **Consultas rápidas mediante índices:** Las bases de datos usan **índices** (similares a un índice de un libro) que aceleran la búsqueda de registros específicos. Consultar un subconjunto de datos en SQL es mucho más veloz que recorrer línea por línea un archivo CSV, especialmente en tablas grandes, porque el motor puede saltar directamente a los datos relevantes.



# ¿Qué ventajas ofrece SQL sobre excel?



- **Relaciones entre datos y menos duplicación:** Con SQL puedes **normalizar** la información en múltiples tablas relacionadas. Por ejemplo, en lugar de tener un gran archivo con datos repetidos, puedes tener una tabla de *Países* y otra de *Indicadores* referenciando al país. Esto evita duplicación y errores. Luego, mediante consultas SQL (*JOIN*), combinas los datos al vuelo.
- **Acceso multiusuario y seguro:** Una base de datos permite que **varios usuarios o aplicaciones accedan simultáneamente** a los datos de forma controlada, sin corromper la información. Por ejemplo, dos analistas pueden consultar o actualizar diferentes partes de la base de datos al mismo tiempo, y el sistema de gestión de bases de datos coordina esos accesos concurrentes de forma segura (transacciones, bloqueos, etc.). En cambio, si varias personas intentan editar un mismo CSV en una red, es probable que surjan conflictos o archivos duplicados.

# ¿Qué ventajas ofrece SQL sobre excel?



- **Integridad y calidad de datos:** Las bases de datos imponen **reglas de validación** (tipos de datos, claves primarias únicas, restricciones *NOT NULL*, etc.) que garantizan la coherencia de la información. Esto evita, por ejemplo, registros duplicados o formatos inválidos (una fecha mal formateada, un campo obligatorio vacío). Un archivo como CSV no impone tales reglas: depende del usuario mantener la limpieza de los datos, lo que es propenso a error.
- **Consultas complejas y funciones agregadas:** Con SQL es muy sencillo obtener derivados de los datos, por ejemplo calculando promedios, sumas, máximos, mínimos agrupados por alguna categoría, todo **mediante una consulta**. Hacer lo mismo en Excel puede implicar fórmulas complejas o *pivots*, y en un CSV requeriría procesar el archivo por completo. SQL ofrece funciones agregadas (*SUM*, *AVG*, *COUNT*, etc.) optimizadas dentro del motor de la base de datos.

# ¿Cómo se usa desde R?



R se integra muy bien con SQL, lo que permite combinar la potencia de análisis estadístico de R con la eficiencia de almacenamiento y consulta de las bases de datos. En la práctica:

- **Conexión a la base de datos:** R dispone del paquete estándar **DBI** (DataBase Interface) que junto con un controlador específico permite conectarse a casi cualquier base de datos. Por ejemplo, usando `DBI::dbConnect()` con el driver adecuado (RPostgres para PostgreSQL, RMySQL/MariaDB, RSQLite, o el genérico ODBC) se establece la conexión desde R a la base de datos. Una vez conectado, R puede enviar consultas SQL o traer tablas completas como si fueran data frames.
- **Ejecución de consultas SQL:** Con la conexión activa, puedes usar funciones como `dbGetQuery()` para **ejecutar sentencias SQL directamente** y obtener el resultado en R. Por ejemplo: `dbGetQuery(con, "SELECT * FROM ventas WHERE region='LATAM'")` te retornaría un data frame en R con los datos filtrados. Así combinas la precisión de SQL para seleccionar datos con la flexibilidad de R para analizarlos.

# ¿Cómo se usa desde R?



R se integra muy bien con SQL, lo que permite combinar la potencia de análisis estadístico de R con la eficiencia de almacenamiento y consulta de las bases de datos. En la práctica:

- **Uso de dplyr con bases de datos:** Alternativamente, R ofrece herramientas como **dplyr** (vía el paquete `dbplyr`) que permiten escribir código R *como si* estuvieras manipulando data frames locales, pero en realidad se traducirá a SQL en segundo plano. Es decir, puedes hacer `ventas_tbl %>% filter(region=="LATAM") %>% summarize(total=sum(monto))` sobre una tabla remota, y dplyr generará la consulta SQL correspondiente automáticamente. Esto simplifica el trabajo para quienes están más familiarizados con la sintaxis de R, a la vez que aprovecha la velocidad de la base de datos.

# ¿Cómo se usa desde R?



R se integra muy bien con SQL, lo que permite combinar la potencia de análisis estadístico de R con la eficiencia de almacenamiento y consulta de las bases de datos. En la práctica:

- **Manejo eficiente de grandes datos:** Un beneficio importante de conectar R a una base SQL es que puedes **procesar los datos en el servidor y traer a R solo los resultados necesarios**. Por ejemplo, en vez de cargar millones de filas a R para luego filtrarlas, le pides a la base de datos que filtre y agregue, y solo importas el resumen. Dado que las tablas SQL se almacenan en disco (y pueden ser enormes) mientras que los *data frames* de R residen en memoria, esta estrategia permite escalar el análisis mucho más allá de la capacidad de RAM de tu equipo.

# ¿Cómo se estructuran los datos en una bd?



Una base de datos relacional organiza la información en **tablas** relacionadas entre sí. Cada tabla es muy similar a una hoja de cálculo o a un *data frame* de R: tiene columnas y filas. Sin embargo, hay conceptos clave en su estructura:

- **Tabla**: conjunto de datos organizado en columnas y filas, que representa una entidad u objeto del mundo real. Por ejemplo, una tabla *Países* podría tener columnas como *código*, *nombre*, *población*, *PIB*, etc., y cada fila corresponde a un país específico. Las tablas en una base de datos relacional están diseñadas para poder vincularse entre sí mediante campos comunes.
- **Fila (registro)**: es cada entrada u observación en la tabla. Corresponde a una instancia de la entidad representada. Siguiendo el ejemplo, una fila de la tabla *Países* contiene todos los datos (código, nombre, población, PIB, ...) de un país específico (ej. una fila para México, otra para Argentina, etc.).

# ¿Cómo se estructuran los datos en una bd?



Una base de datos relacional organiza la información en **tablas** relacionadas entre sí. Cada tabla es muy similar a una hoja de cálculo o a un *data frame* de R: tiene columnas y filas. Sin embargo, hay conceptos clave en su estructura:

- **Columna (campo):** es cada atributo o variable que describe a los registros de la tabla. Todas las filas comparten las mismas columnas. Por ejemplo, la columna *población* en *Países* contendrá el valor de población de cada país. Cada columna tiene un tipo de dato definido (número, texto, fecha, etc.), lo que asegura consistencia en cada campo.
- **Clave primaria:** es una columna (o combinación de columnas) cuyos valores identifican de forma **única** a cada fila de la tabla. No se repiten. Por ejemplo, una columna *codigo\_pais* podría ser la clave primaria en *Países* si cada país tiene un código único. Las claves primarias sirven para poder referenciar un registro de manera inequívoca.



# ¿Cómo se estructuran los datos en una bd?



Una base de datos relacional organiza la información en **tablas** relacionadas entre sí. Cada tabla es muy similar a una hoja de cálculo o a un *data frame* de R: tiene columnas y filas. Sin embargo, hay conceptos clave en su estructura:

- **Clave foránea:** es una columna en una tabla que se refiere (apunta) a la clave primaria de **otra** tabla, estableciendo una relación entre ambas.
- **Esquema:** es el diseño global de la base de datos, que incluye todas sus tablas y las relaciones (claves foráneas) entre ellas. Un buen esquema evita redundancia y asegura integridad: los datos se almacenan *una sola vez* en la tabla correspondiente y se conectan lógicamente mediante relaciones, en lugar de duplicarse en múltiples sitios.



# ¿Qué operaciones se pueden realizar desde SQL?



SQL es muy poderoso y completo; permite tanto **consultar** información como **manipularla** y definir la estructura de la base de datos. Las operaciones principales incluyen:

- **Consultas de datos (*SELECT*):** extraer datos de una o varias tablas. Puedes filtrar filas (*WHERE*), seleccionar columnas específicas, combinar tablas relacionadas (*JOIN*), agrupar resultados (*GROUP BY*) y ordenar (*ORDER BY*), entre otras opciones.
- **Inserción de datos (*INSERT*):** añadir nuevos registros a una tabla. Por ejemplo, insertar un nuevo país en la tabla *Países* con todos sus atributos.
- **Actualización de datos (*UPDATE*):** modificar campos de registros existentes. Por ejemplo, actualizar el valor de población de un país en la tabla *Países* cuando se obtiene el dato nuevo del censo.
- **Eliminación de datos (*DELETE*):** borrar registros que ya no sean necesarios. Por ejemplo, eliminar de una tabla de *Ventas* las transacciones canceladas. (Estas operaciones de modificación pueden restringirse o deshacerse mediante transacciones para seguridad).
- **Control de accesos y seguridad (*DCL – Data Control Language*):** gestionar permisos y usuarios. Por ejemplo, otorgar acceso de solo lectura a ciertos usuarios (*GRANT*) o revocar privilegios (*REVOKE*). Esto asegura que cada usuario vea o edite solo lo que le corresponde.
- Entre otros.

## Ejercicio 02

Descargue la carpeta de la sesión de hoy. Realice las actividades solicitadas en el script Ejercicio 02/ejercicio\_02.R.

## Ejercicio 03

Suponga que usted es un profesor y quiere diseñar una base de datos SQL para guardar todos los datos de sus estudiantes. Elabore cuales serían las tablas que incluiría, cuales serían las llaves primarias y foráneas.

Suponga que tiene una primera versión de las tablas ya hechas en excel: busque en internet como pasar de esas tablas a una base de datos SQL.